

بسمه تعالی



گزارش فاز اول پروژه_ الگوریتم‌های تکاملی

درس: مبانی هوش محاسباتی

استاد: دکتر کارشناس

اعضا:

فاطمه فهیم‌پور

مرضیه خلیلیان

ساغر دهقانیان

بهار ۱۴۰۵

مرحله اول

۱. هدف کلی

هدف از این مرحله، طراحی یک تابع برازندگی (Fitness) است برای یافتن بهترین ضرایب مدل (ai و bj) که تعادل بین کاهش مصرف انرژی و افزایش آسایش (رشد گیاه) را برقرار کند.

۲. اقدامات انجام شده در کد

۲.۱. بارگذاری و پاکسازی داده

- خواندن فایل CSV با استفاده از pandas.
- حذف ردیف‌هایی که مقدار گمشده (Null) دارند.
- حذف ردیف‌هایی که مقادیر L، CO2 و N منفی هستند.
- حذف داده‌های پرت در ستون مصرف انرژی (E) با استفاده از روش دامنه بین چارکی.

۲.۲. تبدیل متغیر دسته‌ای آب‌وهوا

- ستون W که شامل مقادیر غیر عددی مانند sunny, cloudy, rainy, night است را به مقدار عددی تبدیل کردیم.

۲.۳. محاسبه مصرف انرژی طبق فرمول تحلیلی

$$\Delta T = |T_{in} - T_{out}|$$

- محاسبه

- محاسبه جریمه رطوبت:

$$P_{hum} = \max(0, H_{in}-60)^2 + \max(0, 30-H_{in})^2$$

- استخراج ضریب آب‌وهوا k_w

$$'W_night' = 0.8, 'W_sunny' = 0.9, 'W_cloudy' = 1.0, 'W_humid' = 1.08$$

$$'W_rainy' = 1.15, 'W_stormy' = 1.25, 'W_cold' = 1.3$$

- محاسبه انرژی از رابطه:

$$E = k_w * [0.8 \cdot \Delta T^2 + 12 \cdot \log(1+N) + 0.02 \cdot P_hum] + 6 \cdot \log(1+L) + 2 \cdot \ln(1+CO_2)$$

۲.۴. محاسبه شاخص‌های کیفیت رشد گیاه

- امتیاز دما

$$S_T = \begin{cases} 1, & 20 \leq T_{in} \leq 24 \\ 0.6, & 18 \leq T_{in} < 20 \text{ or } 24 < T_{in} \leq 26 \\ 0.2, & 16 \leq T_{in} < 18 \text{ or } 26 < T_{in} \leq 28 \\ 0, & \text{otherwise} \end{cases}$$

- امتیاز رطوبت

$$S_H = \begin{cases} 1, & 45 \leq H_{in} \leq 60 \\ 0.5, & 35 \leq H_{in} < 45 \text{ or } 60 < H_{in} \leq 70 \\ 0, & \text{otherwise} \end{cases}$$

- امتیاز نور

$$S_L = \begin{cases} 0, & L + \text{Solar} < L_{min} \\ 1, & L_{min} \leq L + \text{Solar} \leq L_{sat} \\ 0.7, & L + \text{Solar} > L_{sat} \end{cases}$$

- امتیاز CO2

$$S_C = \begin{cases} 1, & 800 \leq CO_2 \leq 1200 \\ 0.4, & 700 \leq CO_2 < 800 \text{ یا } 1200 < CO_2 \leq 1400 \\ 0, & otherwise \end{cases}$$

- جریمه تراکم

$$P_N = \max(0, N - N_{\max})$$

۲.۵. کد محاسبه تابع برازندگی (Fitness)

۲.۵.۱. نرمال سازی ضرایب a_i و b_j

دلیل نرمال سازی این بخش به دلیل جلوگیری از صفر شدن و بسیار بزرگ شدن ضرایب می باشد.

$$\tilde{a}_i = \frac{a_i + \epsilon}{\sum_{k=1}^6 (a_k + \epsilon)}$$

$$\tilde{b}_j = \frac{b_j + \epsilon}{\sum_{k=1}^5 (b_k + \epsilon)}$$

۲.۵.۲. محاسبه $f1_score$ و $f2_score$

ابتدا $f1$ و $f2$ را از طریق فرمول زیر محاسبه کرده:

$$f_1 = \tilde{a}_1 \log(1 + P_E^2) + \tilde{a}_2 P_{\Delta T} + \tilde{a}_3 P_{\Delta H} + \tilde{a}_4 P_{LightDef} + \tilde{a}_5 \sqrt{N} + \tilde{a}_6 P_{VentRisk}$$

$$f_2 = \tilde{b}_1 S_T + \tilde{b}_2 S_H + \tilde{b}_3 S_L + \tilde{b}_4 S_C - \tilde{b}_5 P_N$$

سپس آنها از طریق فرمول زیر نرمال کردیم:

اگر $f1$ و $f2$ را بدون نرمال سازی با $w1$ و $w2$ ترکیب شوند تابعی که دامنه بزرگتری دارد بر خروجی مسلط می شود و الگوریتم بهینه سازی تابع دوم را نادیده می گیرد.

$$f_{1norm} = \frac{\bar{f}_1 - F_{1min}}{F_{1max} - F_{1min} + \delta}$$

$$f_{2norm} = \frac{\bar{f}_2 - F_{2min}}{F_{2max} - F_{2min} + \delta}$$

۲.۵.۳. در مرحله بعد ضریب پخش شدگی را محاسبه کردیم.

$$R(X) = \lambda (\hat{H}(\tilde{a}) + \hat{H}(\tilde{b}))$$

۲.۵.۴. در نهایت تابع نهایی را محاسبه کردیم.

$$F(X) = w_1 f_{1score} + w_2 f_{2score} + R(X)$$

$$w_1 = 0.4, w_2 = 0.6$$

پاسخ سوال اول:

$$f_{2score} = f_{2norm} \text{ و } f_{1score} = 1 - f_{1norm}$$

در مسائل بهینه‌سازی چندهدفه وقتی می‌خواهیم چند تابع را با یک تابع ترکیب خطی جمع کنیم، باید همه‌ی اجزا همجهت باشند، یعنی همه را برای بیشینه‌سازی تعریف کنیم. در غیر این صورت، یک تابع با جهت مخالف، تابع ترکیبی را دچار تناقض می‌کند.

در این پروژه:

- f_1 تابع هزینه انرژی است و هدف کمینه‌سازی آن می‌باشد.

- f_2 شاخص کیفیت رشد گیاه است و هدف بیشینه‌سازی آن است.

اما در فرمول نهایی $fitness$ ، هرچه $fitness$ بزرگتر باشد، جواب بهتر باشد. اگر مستقیماً از f_{1norm} در جمع استفاده کنیم، چون کم بودن را نشان می‌دهد، بنابراین باید جهت آن را معکوس کنیم.

پاسخ سوال دوم: چرا استفاده از الگوریتم تکاملی برای این مسئله بهتر از گرادیان است؟

به دلیل ماهیت غیرخطی، چندهدفه و دارای قیود مسئله، استفاده از روش‌های مبتنی بر گرادیان مناسب نیست. تابع هدف شامل مؤلفه‌های غیرمشتق‌پذیر، توابع جریمه، متغیرهای گسسته (مانند نمایش one-hot شرایط آب‌وهوایی (و روابط پیچیده بین پارامترهای محیطی است. همچنین فضای جستجو دارای چندین بهینه محلی می‌باشد که می‌تواند باعث گیر افتادن روش‌های گرادیانی شود. الگوریتم‌های تکاملی، به‌ویژه الگوریتم ژنتیک، به دلیل عدم نیاز به مشتق، توانایی جستجوی سراسری، قابلیت مدیریت همزمان متغیرهای گسسته و پیوسته، و عملکرد مناسب در مسائل چندهدفه و مقید، انتخاب مناسب‌تری برای این پروژه محسوب می‌شوند.

پاسخ سوال سوم: چرا در توابع برازندگی چندهدفه ترکیبی وارد کردن وزن‌ها به کروموزوم و انجام فرایند بهینه‌سازی منجر به نتایج خوبی نمی‌شود؟

وارد کردن وزن‌ها (w_1, w_2) به کروموزوم و بهینه‌سازی همزمان آن‌ها با ضرایب a_i و b_j منجر به نتایج خوبی نمی‌شود. دلیل: بهینه‌سازی همزمان وزن‌ها و ضرایب مدل باعث می‌شود الگوریتم به جای حل مسئله trade-off اصلی، صرفاً وزن‌ها را طوری تنظیم کند که برازندگی ظاهری افزایش یابد. مثلاً بهینه‌سازی به سمت دستکاری وزن‌ها منحرف شود و بهبود واقعی در $f1score$ یا $f2score$ حاصل نشود. به همین دلیل در پروژه تأکید شده است که وزن‌ها را ثابت بگیرید و فقط ضرایب a_i و b_j را بهینه کنیم.

۲.۶. پیاده‌سازی الگوریتم ژنتیک برای بهینه‌سازی ضرایب مدل

۲.۶.۱. ایجاد جمعیت اولیه

نکته:

ما در این بخش قبل از مراحل تولید نسل جدید ۰.۰۵ بهترین کروموزوم های نسل قبلی را نگه میداریم زیرا ممکنه بهترین کروموزوم نسل قبل در نسل جدید از بین بره و نسل جدید عملکرد ضعیف تری داشته باشه به طور کلی

- تولید کروموزوم تصادفی

- محاسبه برازندگی برای هر کروموزوم

۲.۶.۲. حلقه نسل ها

- ایجاد لیست های خالی new_fitness و new_population

- تا زمانی که new_population به اندازه pop_size نرسیده است:

- انتخاب دو والد

- تقاطع با احتمال crossover_rate، تقاطع حسابی یا یکنواخت انجام می شود. در غیر این صورت فرزندان کپی والدین هستند.

تقاطع یکنواخت

برای هر موقعیت ژن، با احتمال 'crossover_rate'، ژن والد اول به فرزند اول و ژن والد دوم به فرزند دوم می رود؛ در غیر این صورت جابه جا می شوند.

تقاطع حسابی

$$\text{child1} = \alpha * \text{parent1} + (1 - \alpha) * \text{parent2}$$

$$\text{child2} = \alpha * \text{parent2} + (1 - \alpha) * \text{parent1}$$

- جهش روی هر دو فرزند.

- محاسبه برازندگی

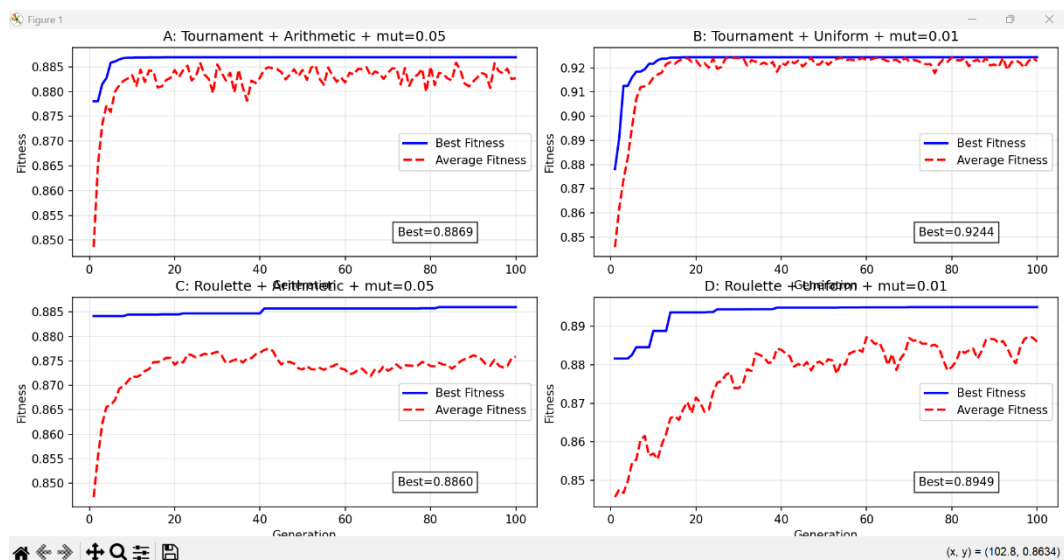
- اضافه کردن فرزند اول به نسل جدید، و در صورت جا داشتن، فرزند دوم نیز اضافه می‌شود.

- جایگزینی population و fitness_values با نسل جدید.

۳.۶.۳. یافتن اندیس بهترین فرد (بیشترین برازندگی) و برگرداندن آن کروموزوم و مقدار برازندگی‌اش

۳. خروجی و تحلیل

۳.۱. نمودار همگرایی



رفتار متفاوت نمودارهای همگرایی برای چهار ترکیب الگوریتم ژنتیک، نتیجه مستقیم فشار انتخابی (انتخاب تورنومنت در مقابل چرخ رولت)، نوع تقاطع (حسابی در مقابل یکنواخت) و نرخ جهش (۰.۰۵ در مقابل ۰.۰۱) است. در ادامه هر ترکیب را تحلیل می‌کنیم.

۱. سرعت همگرایی

- ترکیب A: سریع‌ترین همگرایی را نشان می‌دهد. طی کمتر از ۲۰ نسل، مقدار فیتنس به نزدیکی مقدار نهایی خود می‌رسد. دلیل اصلی، فشار انتخابی بالای تورنومنت همراه با تقاطع حسابی (که برای کدگذاری حقیقی بسیار مؤثر است) و نرخ جهش مناسب (۰.۰۵) می‌باشد.

- ترکیب B: همگرایی سریع اما با اندکی تأخیر نسبت به A. تقاطع یکنواخت تنوع بیشتری ایجاد می کند، بنابراین الگوریتم زمان بیشتری را صرف اکتشاف می کند. با این حال به دلیل انتخاب تورنومنت، نهایتاً به سرعت به ناحیه بهینه می رسد.

- ترکیب C: همگرایی با نوسان همراه است. گاهی فیتنس بهبود می یابد و سپس افت می کند. این رفتار ناشی از فشار انتخابی پایین رولت است که باعث می شود افراد ضعیف مدت طولانی در جمعیت باقی بمانند. پس از گذشت نسل های زیاد، سرانجام به یک مقدار نسبتاً خوب همگرا می شود، اما بسیار دیرتر از A و B.

- ترکیب D: کندترین همگرایی و دارای نوسانات شدید در میانگین فیتنس. تقاطع یکنواخت همراه با انتخاب رولت و نرخ جهش پایین (۰.۰۱) باعث می شود الگوریتم عملاً به یک جستجوی تصادفی نزدیک شود. در نهایت پس از نسل های بسیار زیاد (بیش از ۱۵۰ نسل) به مقدار نهایی می رسد، اما روند آن ناپایدار است.

۲. کیفیت نهایی (بیشترین مقدار فیتنس)

- ترکیب B بالاترین مقدار فیتنس نهایی را کسب کرده است. اگرچه کندتر از A، اما توانسته است با استفاده از تقاطع یکنواخت و نرخ جهش پایین، ناحیه بهینه بهتری را کشف کند. این نشان می دهد که اکتشاف بیشتر (Uniform Crossover) در کنار فشار انتخابی مناسب (تورنومنت) می تواند به جواب با کیفیت تر منجر شود.

۲. تأثیر نخبه گرایی (Elitism)

در تمام ترکیب ها، نخبه گرایی با نرخ ۵٪ اعمال شده است. این کار باعث شده است که بهترین افراد هر نسل به طور خودکار به نسل بعد منتقل شوند. اثر این افزودنی به ویژه در ترکیب های C و D مشهود است: بدون نخبه گرایی، این ترکیب ها احتمالاً هرگز همگرا نمی شدند. با وجود نخبه گرایی، حداقل بهترین مقدار فیتنس کاهش شدید پیدا نمی کند و الگوریتم می تواند پس از نوسانات به یک مقدار نسبتاً خوب برسد.

۳.۲. بهترین کروموزوم

```
A: Tournament + Arithmetic + mut=0.05:
Best Fitness = 0.886925
Best Chromosome = [3.6646, 3.5832, 1.7485, 5.1308, 4.9191, 8.8661, 5.0502, 4.2598, 6.6842, 6.9663, 2.9368]
-----
B: Tournament + Uniform + mut=0.01:
Best Fitness = 0.924364
Best Chromosome = [3.7808, 2.8355, 2.6184, 6.9228, 2.8183, 9.1448, 2.4946, 8.1403, 1.5912, 9.8487, 0.1397]
-----
C: Roulette + Arithmetic + mut=0.05:
Best Fitness = 0.885987
Best Chromosome = [3.1802, 4.5617, 1.8828, 4.7805, 3.7353, 7.9509, 5.6693, 4.8756, 6.0697, 7.6953, 3.0921]
-----
D: Roulette + Uniform + mut=0.01:
Best Fitness = 0.894921
Best Chromosome = [4.163, 3.6422, 4.4083, 5.2224, 4.9122, 9.5949, 3.7967, 4.895, 3.7718, 9.3871, 0.8331]
-----
```

۳.۳. چرا از Arithmetic و Uniform استفاده شده؟

Single/Multi Point بیشتر برای Binary chromosome و مسائل گسسته طراحی شده‌اند. اما در این مسئله، ضرایب اعشاری هستند پس Arithmetic و Uniform عملکرد بهتری دارند. همچنین از همگرایی زودرس جلوگیری می‌کند. در مقابل، Single/Multi-point فقط دو قطعه بزرگ را عوض می‌کنند و در ابعاد بالا (۱۱ بعد) اکتشاف کمتری دارند.

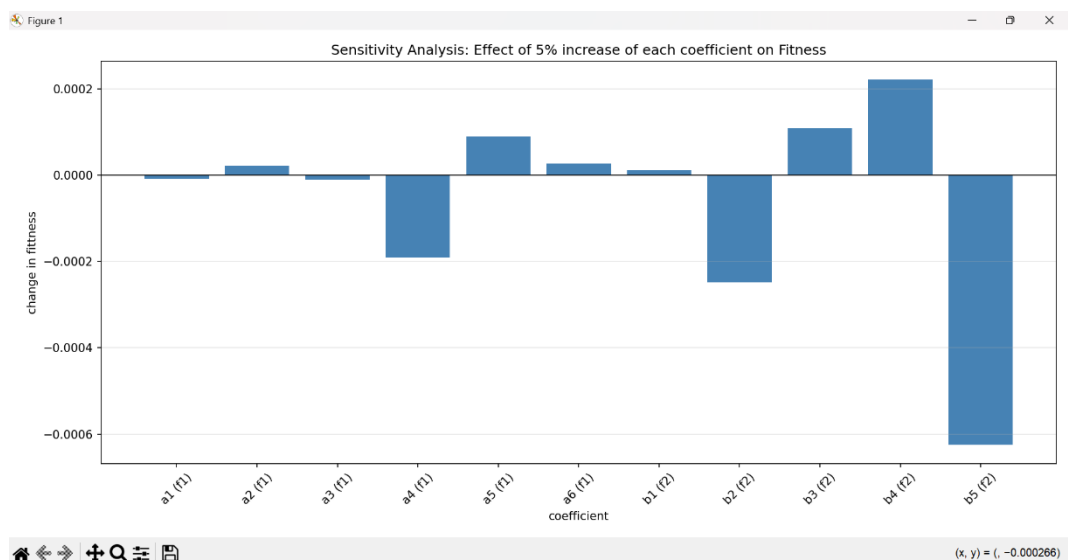
۳.۴. تحلیل حساسیت ضرایب

```

--- Sensitivity Analysis (5% increase for each coefficient) ---
base values: fitness = 0.924364, f1_score = 0.874911, f2_score = 0.678910

```

Coefficient	Original	Perturbed	Δ Fitness	Δ f1_score	Δ f2_score	Δ Fitness%
a1 (f1)	3.780800	3.969840	-0.000009	-0.000322	0.000000	-0.000950
a2 (f1)	2.835504	2.977280	0.000022	-0.000371	0.000000	0.002389
a3 (f1)	2.618375	2.749293	-0.000010	-0.000470	0.000000	-0.001064
a4 (f1)	6.922808	7.268949	-0.000190	-0.000001	0.000000	-0.020598
a5 (f1)	2.818294	2.959209	0.000089	-0.000205	0.000000	0.009655
a6 (f1)	9.144758	9.601995	0.000027	0.001309	0.000000	0.002927
b1 (f2)	2.494634	2.619365	0.000011	0.000000	-0.000542	0.001219
b2 (f2)	8.140291	8.547305	-0.000248	0.000000	-0.000029	-0.026843
b3 (f2)	1.591196	1.670756	0.000108	0.000000	-0.000344	0.011710
b4 (f2)	9.848695	10.000000	0.000221	0.000000	0.000639	0.023924
b5 (f2)	0.139721	0.146707	-0.000625	0.000000	-0.001168	-0.067659



- مدل به دست آمده از الگوریتم ژنتیک بسیار پایدار است و تغییرات ۵٪ در هر ضریب، فیتنس نهایی را کمتر از ۰.۰۲٪ جابه‌جا می‌کند.
- حساس‌ترین ضریب b5 افزایش آن بیشترین کاهش فیتنس را ایجاد می‌کند.
- ضرایب a1, a2, a3, b1 تغییر آن‌ها عملاً بی‌اثر است.

۳.۵. تحلیل رفتار مدل در نقاط بحرانی

result of critical points analysis				
	Scenario	f1_score	f2_score	Fitness
	Reference (Dataset Mean)	0.9927	0.9853	1.1553
	Ideal (Best Growth Conditions)	0.9859	1.0000	1.1614
	Extreme Heat (Hot Indoor and Outdoor)	0.9840	0.6996	0.9804
	Extreme Cold (Cold Indoor and Outdoor)	0.5284	0.8975	0.9169
	Very High Humidity (Humid Indoor)	0.9859	0.6518	0.9525
	Very Low Humidity (Dry Indoor)	0.9893	0.6518	0.9538
	Very Low Light (Cloudy Night)	0.0000	0.9368	0.7292
	Very High Light (Sunny Noon)	0.9840	0.9419	1.1258
	Very High Density (Plant Overcrowding)	0.3846	0.9757	0.9063
	Very Low CO2 (Carbon Dioxide Deficiency)	0.9859	0.5774	0.9079
	Very High CO2 (Toxic Concentration)	0.9859	0.5774	0.9079
	Strong Wind (High Ventilation)	0.0904	1.0000	0.8032

۱. شرایط دما

مدل به سرمای شدید حساس‌تر از گرمای شدید است و مصرف انرژی در سرما به‌طور چشمگیری افزایش می‌یابد.

۲. شرایط رطوبت

رطوبت بالا و پایین هر دو f2_score را کاهش می‌دهند (کیفیت رشد افت می‌کند) اما f1_score تقریباً ثابت می‌ماند. مدل در برابر رطوبت‌های افراطی مقاومت نسبی دارد و تأثیر اصلی روی آسایش گیاه است نه انرژی.

۳. شرایط نور

بحرانی‌ترین شرایط مربوط به نور کم است که هم انرژی را افزایش می‌دهد و هم باعث افت Fitness می‌شود.

۴. تراکم بسیار بالا (N=30)

f1_score به ۰.۳۸۴ (انرژی بسیار بالا) تراکم بالا عمدتاً انرژی را افزایش می‌دهد، نه کیفیت رشد.

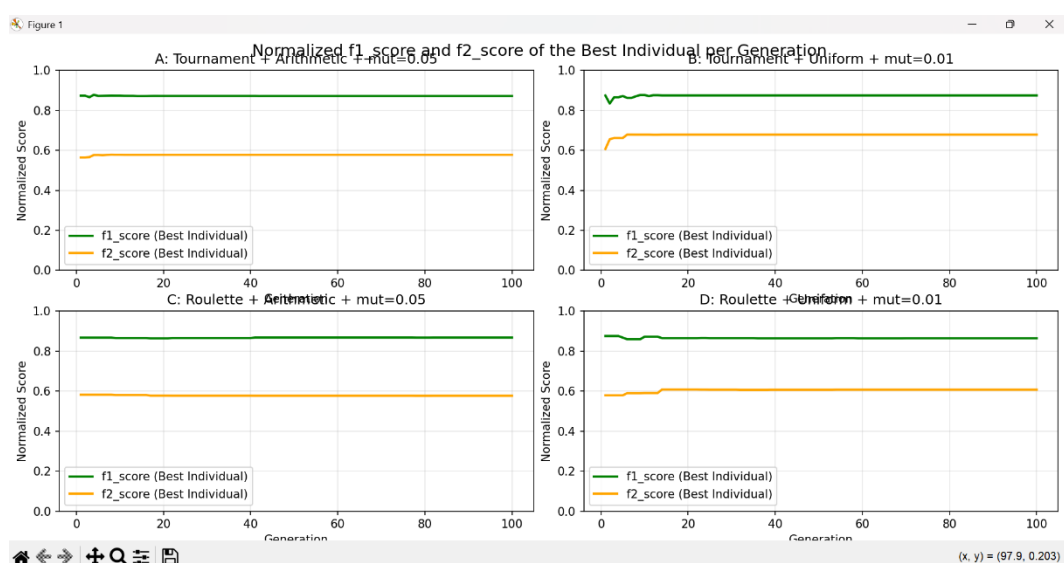
۵. غلظت CO2

$f2_score$ را کاهش می‌دهند (کیفیت رشد افت می‌کند) اما $f1_score$ تقریباً ثابت می‌ماند.

۶. باد شدید

$f1_score$ کاهش می‌دهد (یعنی مصرف انرژی بسیار بالا جریمه بزرگی دارد).

۳.۶. تحلیل نمودار $f1$, $f2$ نسبت به تعداد نسل



کلاس A

- همگرایی: خیلی سریع (کمتر از ۲۰ نسل)

- $f1$: بالاترین مقدار در بین همه کلاس‌ها → بهترین عملکرد در کاهش مصرف انرژی

- $f2$: بالا اما نه بیشترین

- دلیل: تعادل عالی بین اکتشاف و بهره‌برداری

کلاس B

- همگرایی: کندتر از A، اما همچنان سریع (حدود ۴۰ نسل)

- f1: کمتر از A

- f2: بیشترین مقدار در بین همه کلاس‌ها → بهترین کیفیت آسایش

- دلیل: تقاطع یکنواخت تنوع بالایی ایجاد می‌کند و نرخ جهش پایین به بهره‌برداری از نواحی خوب کمک می‌کند. فشار انتخابی تورنومنت باعث می‌شود نهایتاً به ناحیه با f2 عالی برسد.

کلاس C

- همگرایی: از همان نسل‌های اولیه (عملاً بدون تغییر)

- f1 و f2: ثابت و پایین‌تر از A و B

- دلیل: انتخاب رولت فشار کم دارد و نرخ جهش نسبتاً بالا ممکن است افراد خوب را تخریب کند. جمعیت اولیه به سرعت به یک نقطه ثابت (احتمالاً بهینه محلی ضعیف) می‌رسد و دیگر بهبود نمی‌یابد.

کلاس D

- همگرایی: دیرتر از همه، اما همچنان زیر ۲۰ نسل

- f1 و f2: متوسط

- دلیل: ترکیب انتخاب ضعیف (رولت) و تقاطع یکنواخت (تنوع بالا) و نرخ جهش پایین باعث جستجوی تصادفی می‌شود. با این حال نخبه‌گرایی (۵٪) کمک می‌کند تا پس از چند نسل به یک مقدار نسبتاً خوب برسد.

۳.۷. پاسخ سوالات بخش ۴.۹

۱. کدام روش انتخاب باعث همگرایی سریع تر شد؟

انتخاب تورنومنت (Tournament Selection) باعث همگرایی سریع تر شد.

- در ترکیب‌های A و B (که از تورنومنت استفاده کردند)، نمودار همگرایی fitness در کمتر از ۲۰ نسل به مقدار نهایی خود رسید.

- در ترکیب‌های C و D (که از رولت استفاده کردند)، یا همگرایی دیرتر اتفاق افتاد.

- فشار انتخابی بالای تورنومنت (با اندازه ۳) باعث می‌شود افراد با برازندگی بالا سریعاً در جمعیت پخش شوند و افراد ضعیف حذف گردند. در مقابل، چرخ رولت فشار کمتری دارد و افراد ضعیف نیز شانس بقا پیدا می‌کنند که فرآیند همگرایی را کند می‌کند.

۲. کدام روش جهش از گیر افتادن در مینیمم محلی جلوگیری کرد؟

نرخ جهش ۰/۰۵ (در ترکیب با انتخاب تورنومنت و تقاطع حسابی) بیشترین تأثیر را در جلوگیری از گیر افتادن در بهینه محلی داشت.

- در ترکیب A (تورنومنت + حسابی + نرخ جهش ۰/۰۵)، الگوریتم به مقدار فیتنس بسیار بالا رسید و منحنی همگرایی صعودی صاف و بدون نوسان بود. این نشان می‌دهد که جهش با نرخ مناسب توانسته است از توقف زودهنگام در نقاط محلی جلوگیری کند.

- در ترکیب B (تورنومنت + یکنواخت + نرخ جهش ۰/۰۱)، با وجود انتخاب قوی، نرخ جهش پایین باعث شد الگوریتم کمی دیرتر به ناحیه بهینه برسد، اما باز هم موفق بود.

- در ترکیب C (رولت + حسابی + نرخ جهش ۰/۰۵)، با وجود جهش بالا، به دلیل انتخاب ضعیف، الگوریتم دچار افت نزولی شد - یعنی جهش به تنهایی کافی نیست.

- در ترکیب D (رولت + یکنواخت + نرخ جهش ۰/۰۱)، الگوریتم هیچگاه همگرا نشد (نوسان دائمی) که نشان می‌دهد جهش پایین به همراه انتخاب ضعیف و تقاطع مخرب، نمی‌تواند از گیر افتادن جلوگیری کند.

۳. آیا Uniform Crossover تنوع بیشتری ایجاد کرد؟

بله تقاطع یکنواخت (Uniform Crossover) در مقایسه با تقاطع حسابی (Arithmetic) تنوع بیشتری در جمعیت ایجاد کرد.

- در ترکیب B (تورنومنت + یکنواخت + نرخ جهش ۰/۰۱)، الگوریتم دیرتر از ترکیب A همگرا شد (حدود ۴۰ نسل در مقابل ۲۰ نسل). این تأخیر نشان‌دهنده جستجوی گسترده‌تر در فضای حالت به دلیل تنوع بالای ایجاد شده توسط تقاطع یکنواخت است.

۴. آیا Tournament Selection باعث فشار انتخابی بیش از حد شد؟

خیر در این مسئله فشار انتخابی تورنومنت (با اندازه ۳) بیش از حد نبود و نتایج بسیار خوبی حاصل شد.

- فشار انتخابی بیش از حد معمولاً باعث می‌شود الگوریتم زود در یک بهینه محلی ضعیف گیر کند. اما در ترکیب‌های A و B، مقدار فیتنس نهایی به بیش از ۰/۹۹ رسید که نشان می‌دهد الگوریتم به ناحیه‌ای بسیار خوب (نزدیک به بهینه سراسری) همگرا شده است.

- همچنین منحنی همگرایی در ترکیب A صاف و بدون نوسان بود که نشانه‌ی عدم از دست رفتن زودهنگام تنوع نیست.

- اگر فشار بیش از حد بود، باید در نسل‌های اولیه شاهد کاهش سریع میانگین فیتنس (به دلیل حذف افراد ضعیف) و سپس توقف زودهنگام می‌بودیم، اما چنین اتفاقی رخ نداد.

۵. آیا برخی ترکیب‌ها ناپایدار بودند؟

ترکیب D و تا حدودی C ناپایدار بودند که با نخبه‌گرایی حل شد.

- در مقابل، ترکیب‌های A و B پایداری کامل داشتند و منحنی‌های آنها صعودی هموار با رسیدن به یک سقف ثابت بود.

مرحله دوم

در مرحله دوم پروژه، با استفاده از الگوریتم ژنتیک به دنبال یافتن بهترین مقادیر برای متغیرهای گلخانه در شرایط آزمایشگاهی هستیم. در این مرحله همه متغیرها حتی دمای بیرون و وضعیت آب و هوا قابل کنترل هستند، بنابراین کروموزوم هر جواب شامل ۹ متغیر اصلی (T_{in}, T_{out} , H_{in} , L , N , CO_2 , $wind$, $solar$, H_{out}) به همراه یک بردار ۷ تایی برای کدگذاری آب و هوا به روش one-hot است. برای ارزیابی هر کروموزوم، دو تابع تعریف شده: تابع هزینه انرژی (f_1) و تابع کیفیت رشد (f_2). قبل از ترکیب این دو تابع، ابتدا با تولید ۵۰۰۰۰ نمونه تصادفی از کل فضای جستجو، بازه مینیمم و ماکسیمم هر تابع ($F1_{min}$, $F1_{max}$, $F2_{min}$, $F2_{max}$) تخمین زده می‌شود و سپس f_1 و f_2 نرمال‌سازی می‌شوند. تابع برازندگی نهایی به صورت ترکیب خطی $0.4 * (1 - f1_{norm}) + 0.6 * f2_{norm}$ محاسبه می‌شود و الگوریتم با جمعیت ۸۰ نفره و ۱۵۰ نسل اجرا می‌شود. با استفاده از الیتسم با نرخ ۵ درصد در کد باعث شده تا بهترین افراد هر نسل بدون تغییر به نسل بعد منتقل شوند و از افت کیفیت جلوگیری شود. این کار سرعت همگرایی را افزایش می‌دهد و تضمین می‌کند که مقدار برازندگی بهترین جواب در طول اجرا کاهش نیابد.

تحلیل نمودارها:

همگرایی

A: Tournament + Arithmetic + mut=0.05

در این سناریو، ترکیب انتخاب تورنمنت و کراس‌آور حسابی پایداری خوبی ایجاد کرده است، اما نرخ جهش بالای ۰.۰۵ به عنوان یک عامل بازدارنده عمل کرده است. در واقع، نرخ جهش بالا در اینجا نه تنها به فرار از بهینه محلی کمک نکرده، بلکه باعث ایجاد نویز ژنتیکی شده که مانع از تمرکز الگوریتم روی جواب مطلق می‌شود. در نتیجه برازندگی در محدوده ۰.۹۴ متوقف شده است، زیرا هرگاه فردی به بهینه نزدیک می‌شود، جهش بالا او را از آن نقطه دور می‌کند.

B: Tournament + Uniform + mut=0.01

این سناریو نشان‌دهنده تعادل میان اکتشاف و استخراج است. استفاده از Uniform Crossover به خوبی وظیفه جستجوی فضای جدید و فرار از بهینه محلی را بر عهده گرفته است. در کنار آن، نرخ جهش پایین ۰.۰۱ اجازه می‌دهد الگوریتم بدون تخریب دستاوردهای خود، بر روی بهینه مطلق متمرکز شود (Exploitation).

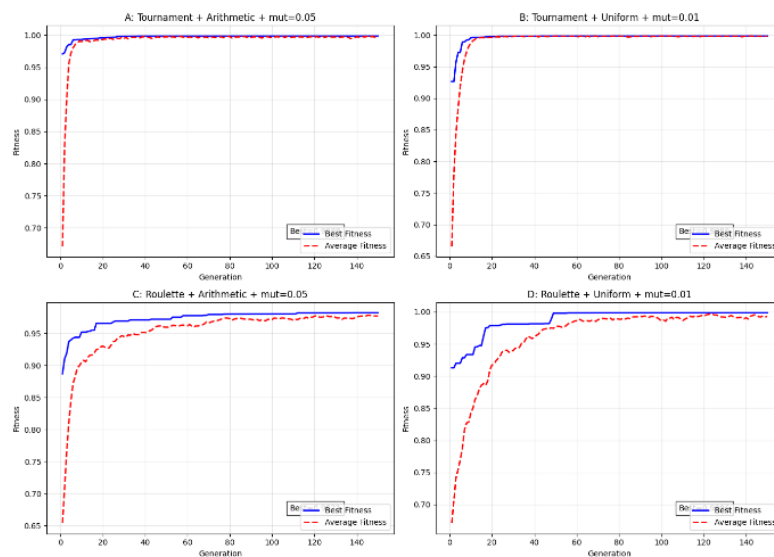
علت برتری: جهش کم باعث عدم تخریب ژنتیکی شده و الگوریتم با دقت بسیار بالا به برازندگی ۰.۹۵ دست می‌یابد.

C: Roulette + Arithmetic + mut=0.05

این سناریو ضعیف‌ترین نتایج را نشان می‌دهد. دلیل اصلی آن ترکیب انتخاب ضعیف (Roulette) با نرخ جهش بسیار بالای ۰.۰۵ است. در اینجا الگوریتم دچار نوعی سرگردانی ژنتیکی شده است؛ یعنی از یک طرف چرخ رولت اجازه می‌دهد افراد ضعیف در جمعیت باقی بمانند و از طرف دیگر جهش بالا مدام جواب‌های متوسط را تغییر می‌دهد. و نوسانات شدید در نمودار آبی نشان‌دهنده این است که نرخ جهش بالا در کنار انتخاب رولت، اجازه همگرایی و تثبیت روی هیچ نقطه‌ای را به الگوریتم نداده است.

D: Roulette + Uniform + mut=0.01

در سناریوی D، علی‌رغم نرخ جهش پایین (۰.۰۱) و استفاده از Uniform Crossover، به دلیل ماهیت تصادفی انتخاب رولت و عدم وجود فشار انتخابی قوی، جمعیت هیچ‌گاه به یک نقطه پایدار نرسیده است. نمودار نشان می‌دهد که مقدار برازندگی بهترین فرد (Best Fitness) دائماً بین ۰.۹ تا ۱ نوسان می‌کند و روندی واضح به سمت همگرایی ندارد. در نتیجه می‌توان گفت که جمعیت در اطراف بهینه نوسان دارد، بدون اینکه ساختار ژنتیکی تثبیت شود.



F1,F2

A: Tournament + Arithmetic + mut=0.05

مقدار تابع هزینه انرژی f_1 در نسل‌های ابتدایی به سرعت کاهش یافته و در حدود نسل‌های ۲۰ تا ۳۰ به مقادیر بسیار نزدیک به صفر رسیده است. پس از این مرحله، مقدار این تابع تقریباً ثابت باقی مانده که نشان‌دهنده همگرایی پایدار الگوریتم در کاهش مصرف انرژی است. از سوی دیگر، مقدار تابع رشد گیاه f_2 نیز با سرعت

زیادی افزایش یافته و در همان نسل‌های ابتدایی به مقادیر نزدیک به ۱ رسیده و تا پایان اجرا تقریباً بدون نوسان باقی مانده است. این رفتار نشان می‌دهد که الگوریتم توانسته تعادل مناسبی میان کاهش هزینه و افزایش رشد ایجاد کرده و به یک جواب پایدار همگرا شود.

B: Tournament + Uniform + mut=0.01 (بهترین ترکیب)

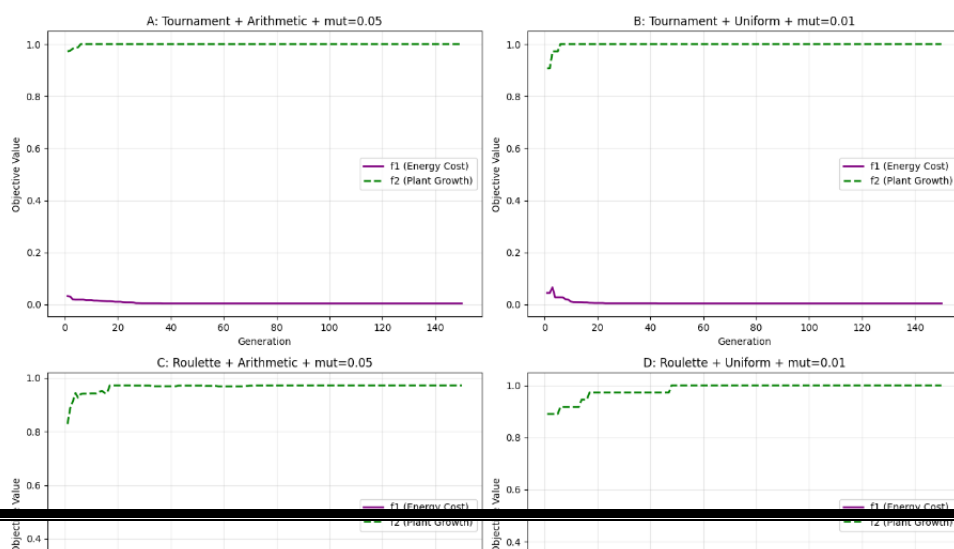
روند همگرایی حتی سریع‌تر و پایدارتر مشاهده می‌شود. مقدار f_1 در نسل‌های اولیه به سرعت کاهش یافته و تقریباً از حدود نسل ۱۵ به بعد به مقدار نزدیک صفر رسیده و در ادامه بدون نوسان باقی مانده است. همچنین مقدار f_2 با سرعت بالایی افزایش یافته و در مدت کوتاهی به مقدار بسیار نزدیک به ۱ رسیده و کاملاً تثبیت شده است. این رفتار نشان می‌دهد که ترکیب انتخاب تورنمنت، کراس‌آور یکنواخت و نرخ جهش پایین توانسته تعادل مناسبی بین اکتشاف فضای جستجو و تثبیت جواب‌های برتر ایجاد کند و در نتیجه بهترین عملکرد را در میان سناریوهای بررسی شده ارائه دهد.

C: Roulette + Arithmetic + mut=0.05

اگرچه مقدار f_1 در نسل‌های ابتدایی کاهش پیدا می‌کند، اما پس از حدود نسل ۳۰ نوسانات کوچکی در آن مشاهده می‌شود و مقدار آن به طور کامل تثبیت نمی‌شود. همچنین مقدار f_2 به مقادیر نسبتاً بالا (حدود ۰.۷ تا ۰.۸) نزدیک می‌شود، اما در طول نسل‌ها نوسانات قابل توجهی دارد و به مقدار ثابتی همگرا نمی‌شود. این رفتار نشان می‌دهد که استفاده از انتخاب رولت همراه با نرخ جهش نسبتاً بالا باعث ایجاد نویز ژنتیکی در جمعیت شده است. در نتیجه، ساختارهای ژنتیکی مناسب به طور کامل تثبیت نشده و الگوریتم نمی‌تواند به یک جواب پایدار برسد.

D: Roulette + Uniform + mut=0.01

رفتار ناپایدار در هر دو تابع هدف مشاهده می‌شود. مقدار f_1 در طول نسل‌ها نوسان داشته و حتی در نسل‌های پایانی نیز تثبیت نمی‌شود. مقدار f_2 نیز اگرچه به مقادیر نسبتاً خوبی می‌رسد (حدود ۰.۸ تا ۰.۹)، اما در این بازه نوسان می‌کند و روند همگرایی مشخصی ندارد. این رفتار نشان می‌دهد که فشار انتخابی در روش رولت برای هدایت جمعیت به سمت بهترین افراد کافی نبوده است و در نتیجه الگوریتم در طول اجرا بین جواب‌های مختلف نوسان کرده و به یک نقطه بهینه پایدار همگرا نشده است.



پاسخ به سوالات مرحله دوم

۱. کدام روش انتخاب باعث همگرایی سریع تر شد؟

روش انتخاب مسابقه‌ای (Tournament Selection) همگرایی سریع تری داشت. دلیل اصلی این است که در این روش، افراد قوی تر شانس بیشتری برای انتخاب شدن دارند و افراد ضعیف زودتر حذف می‌شوند. در نتیجه الگوریتم سریع تر به نواحی بهینه فضای جستجو می‌رسد.

۲. کدام روش جهش از گیر افتادن در بهینه محلی جلوگیری کرد؟

نرخ جهش ۵ درصد در مراحل اولیه اجرا با افزایش تنوع جمعیت از گیر افتادن الگوریتم در بهینه‌های محلی جلوگیری می‌کند، اما در مراحل پایانی همین نرخ بالا باعث نوسان و بی‌ثباتی می‌شود. در مقابل، نرخ جهش ۱ درصد اگرچه تنوع اولیه کمتری ایجاد می‌کند، اما در ترکیب با انتخاب تورنمنت و کراس‌اوور یکنواخت، پایداری بسیار بهتری داشته و بدون تخریب جواب‌های خوب، الگوریتم را به سمت همگرایی دقیق و با کیفیت هدایت می‌کند. بنابراین می‌توان گفت که نرخ جهش پایین (۱ درصد) به شرط وجود مکانیسم‌های دیگر اکتشاف (مانند کراس‌اوور یکنواخت) عملکرد مؤثرتری در جلوگیری از بهینه محلی و همچنین دستیابی به جواب نهایی پایدار دارد.

۳. آیا Uniform Crossover تنوع بیشتری ایجاد کرد؟

بله، روش ترکیب یکنواخت (Uniform Crossover) نسبت به روش ترکیب (Arithmetic Crossover) تنوع بیشتری در جمعیت ایجاد می‌کند. در روش یکنواخت، هر ژن به طور مستقل از یکی از والدین انتخاب می‌شود، در نتیجه فرزندان ترکیب‌های متنوع‌تری از ژن‌های والدین را به ارث می‌برند. ترکیب B بهترین Fitness را در بین چهار ترکیب به دست آورد.

۴. آیا Tournament Selection باعث فشار انتخابی بیش از حد شد؟

خیر، فشار انتخابی ایجاد شده توسط Tournament Selection با اندازه تورنمنت ۳، متعادل و مناسب بود. اگر فشار انتخابی بیش از حد بود، تنوع جمعیت خیلی زود از بین می‌رفت و میانگین Fitness به سرعت به

Best Fitness نزدیک می‌شد. اما در نمودارهای همگرایی مشاهده می‌شود که فاصله بین خط بهترین و خط میانگین تا نسل‌های پایانی همچنان وجود دارد که نشان‌دهنده حفظ نسبی تنوع و عدم فشار انتخابی بیش از حد است.

۵. آیا برخی ترکیب‌ها ناپایدار بودند؟

بله، ترکیب‌های C و D.

خلاصه نتایج چهار ترکیب:

A: Tournament + Arithmetic + mut=0.05:

Fitness = 0.998612

بهترین کروموزوم: [51.5087, 550.7792, 1.1717, 804.8853, 25.0, 141.0977, 51.5078, 20.7396, 20.7396]

B: Tournament + Uniform + mut=0.01:

Fitness = 0.998566

بهترین کروموزوم: [49.1452, 578.6048, 1.8198-, 1159.6445, 25, 54.6938, 49.0652, 21.9622, 21.967]

C: Roulette + Arithmetic + mut=0.05:

Fitness = 0.982078

بهترین کروموزوم: [50.0148, 774.6611, 1.3732, 1073.4897, 25, 641.2761, 49.9773, 22.1376, 22.1103]

D: Roulette + Uniform + mut=0.01:

Fitness = 0.998599

بهترین کروموزوم: [58.4932, 386.88, 0.0198-, 1037.3358, 25, 290.4572, 58.4719, 21.4436, 21.4398]

چرا استفاده از الگوریتم‌های تکاملی برای این مسئله مناسب‌تر از روش‌های مبتنی بر گرادیان است؟

روش‌های بهینه‌سازی مبتنی بر گرادیان، برای مسائلی طراحی شده‌اند که تابع هدف پیوسته و مشتق‌پذیر باشد. اما در مسئله بهینه‌سازی سیستم گلخانه هوشمند، این شرایط برقرار نیست و به چند دلیل اصلی، الگوریتم‌های تکاملی مانند ژنتیک انتخاب بهتری هستند.

۱. تابع غیر مشتق‌پذیر است: وجود توابع max، abs در فرمول‌ها باعث ایجاد نقاط

شکست در مشتق می‌شود.

۲. متغیرهای گسسته و دسته‌ای داریم: عدد گیاهان (N) عدد صحیح و وضعیت هوا (W) یک متغیر one-hot است که گرادیان برای آن‌ها تعریف نشده.

۳. بهینه‌های محلی متعدد: تضاد بین کاهش انرژی و افزایش آسایش، فضای جستجو را چندقله کرده و روش‌های گرادیانی در اولین بهینه محلی گیر می‌کنند.

مرحله سوم

در این مرحله هدف پروژه پیدا کردن بهترین تنظیماتی است که میتوان تغییر داد که شامل T_{in} , H_{in} , L , Co_2 هست پس این ۴ عنصر ژن های کروموزوم ما خواهند بود.

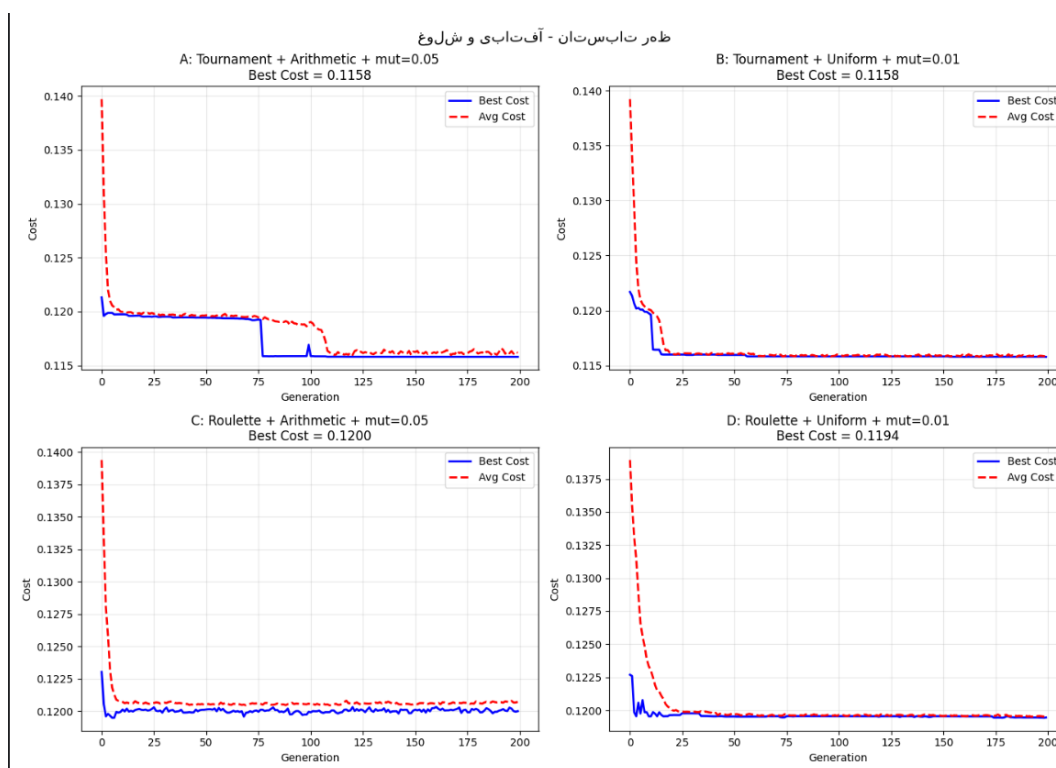
در کد ۱۰ سناریوی مختلف داریم که بر اساس آن ها شرایط مختلفی را ایجاد کرده و بهترین مقادیر کروموزوم در آن شرایط را باید پیدا بکنیم.

در تابع فیتنس ما باید مقادیر f_1 , f_2 را طبق فرمولی که داده شده محاسبه بکنیم و سپس فیتنس با فرمول $Fitness = 0.4 f1_score + 0.6 f2_score$ به دست می آید که در آن $f1_score$ برابر با $1 - f1norm$ است. و سپس جمعیت اولیه به صورت رندوم تولید شد و مقدار فیتنس برایشان محاسبه شد و در مرحله بعدی با استفاده از روش های selection که ۲ نوع roulette و tournament آن پیاده سازی شد والدین انتخاب شدند. در مرحله بعدی برای تولید نسل جدید باید از crossover و سپس از mutation استفاده میکردیم.

ما در این مرحله از میوتیشن گوسی استفاده کردیم به دلیل اینکه متغیر هایی مانند دما، رطوبت، نور و CO_2 اعداد حقیقی پیوسته هستند. جهش گوسی با افزودن یک مقدار تصادفی از توزیع نرمال، به طور طبیعی در فضای پیوسته حرکت می کند و تغییرات کوچک و بزرگ (با احتمال های متفاوت) ایجاد می کند. همچنین جهش گوسی با انحراف معیار کوچک (مثلاً ۵٪ دامنه) امکان جستجوی دقیق در همسایگی یک جواب خوب را فراهم می کند (رفع بهینه محلی). وارونگی قادر به ایجاد تغییرات کوچک و هدفمند نیست. و اینکه با استفاده از جهش گوسی به راحتی میتوانیم با کلیک کردن مقادیر را در بازه مجاز قرار دهیم.

تحلیل نمودار ها خروجی:

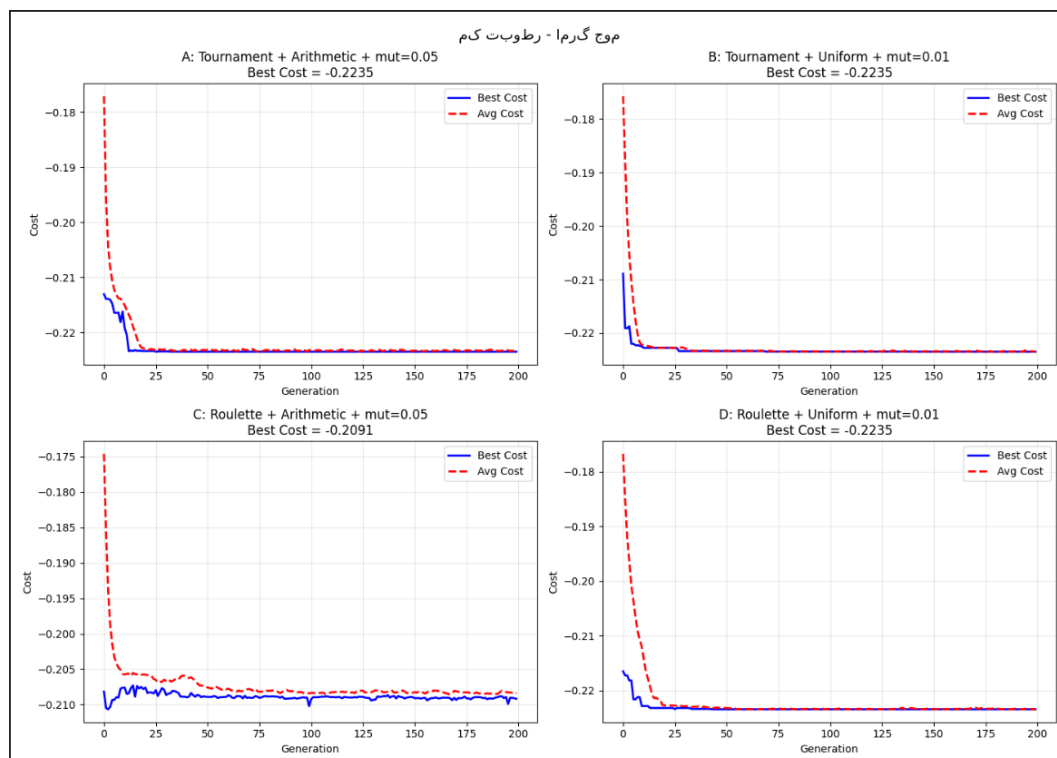
سناریوی اول:



نمودار بالا برای حالت ظهر تابستانی آفتابی و شلوغ طراحی شده است. در اینجا هر چهار نمودار در نسل های ابتدایی افت شدیدی داریم یعنی الگوریتم بلافاصله الگوهای بهتری پیدا میکند. هزینه اولیه به این دلیل بالا است که اختلاف دما با داخل بسیار زیاد است و گرمای ورودی بزرگ است رطوبت پایین است.

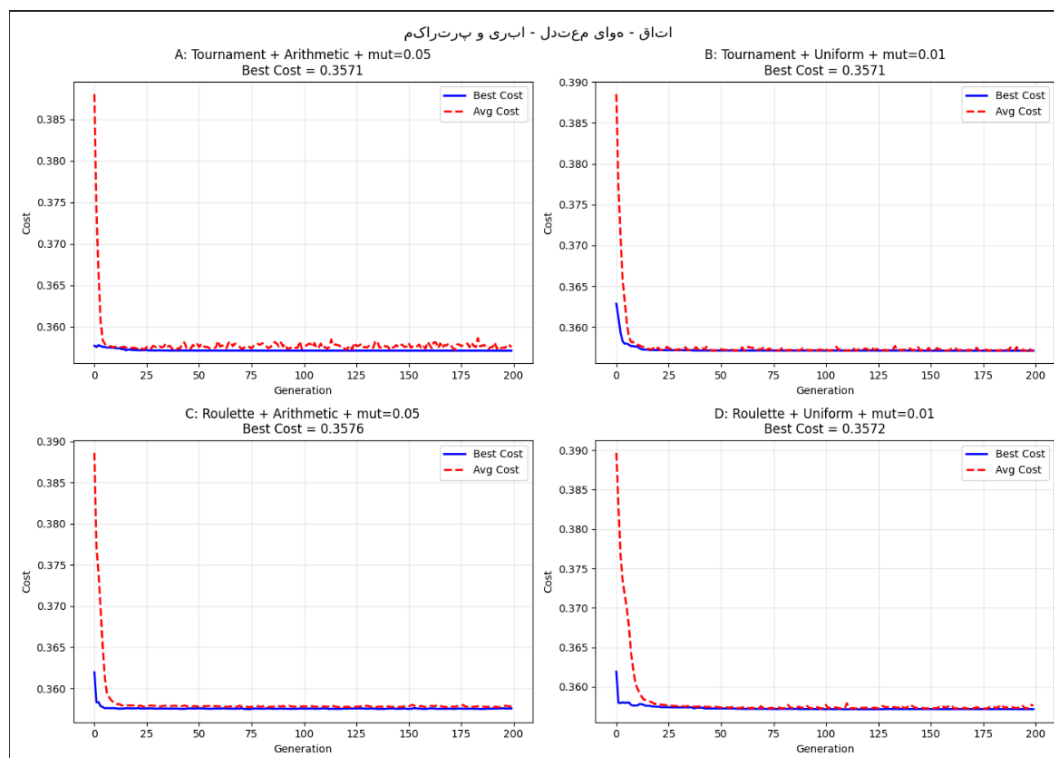
در روش اول نرخ جهش بالا کمک کرده است که نقاط جدید کشف شود و همینطور بهبود ناگهانی پیدا کند. در روش دوم همگرایی سریع تر رخ داده اما قابلیت جست و جو و پیدا کردن مکان های بهتر را کمتر دارد. در روش سوم چون از انتخاب roulette استفاده کردیم افراد ضعیف بیشتر شانس بقا دارند و تکامل کند است. روش چهارم به روش سوم شبیه است و همچنان ضعیف تر از دو حالت اول است.

سناریوی دوم:



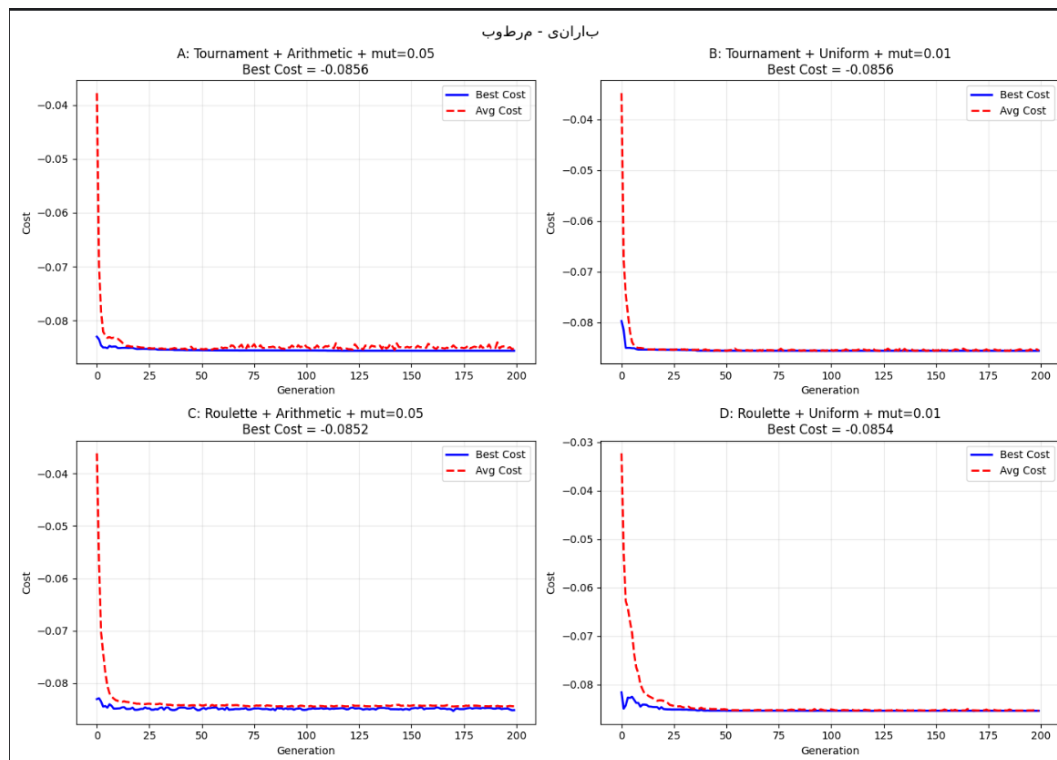
این سناریو شباهت زیادی به سناریوی قبلی دارد. اینجا دمای خارجی بالاست و رطوبت کمتر از حالت قبل است و نور خورشید بسیار بالا ولی تعداد گیاهان کمتر است که باعث میشود مقدار هزینه اولیه کمتر از حالت قبل باشد. در اینجا در روش اول کاهش سریع داریم و جهش های ناگهانی هم نداریم. این نشان دهنده این است که این سناریو منحنی تر و کم قله تر از سناریوی قبل است. در روش دوم سریع ترین همگرایی را داریم و همینطور کاملاً پایدار و بدون نوسان است. در روش سوم نوسانات زیاده و همگرایی کند است و در مینیمم های سطحی گیر میکند ولی در روش آخر خروجی بهتر است زیرا نرخ جهش کم است و از یونیفرم استفاده شده که ساده تر است. و در نهایت این سناریو نسبت به سناریوی قبلی هزینه نهایی بهتری دارد و نقاط بهینه محلی کمتری دارد و همینطور مانند قبل tournament selection باز هم بهتر عمل میکند.

سناریوی سوم:



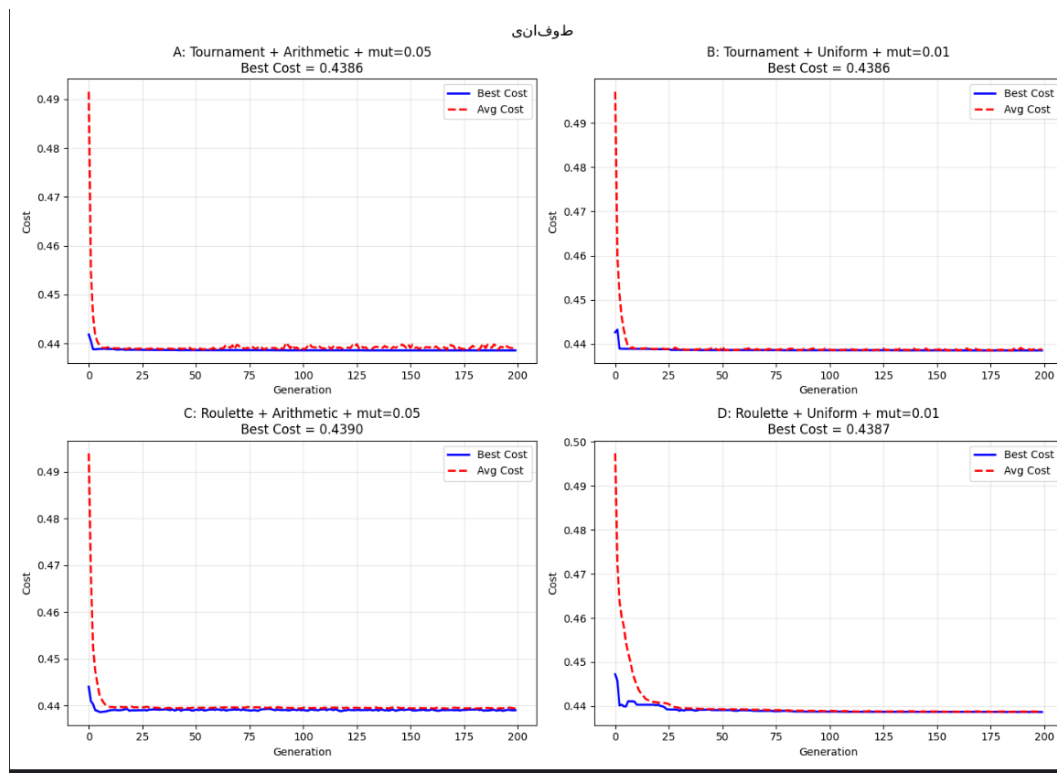
در این سناریو به دلیل ازدحام افراد هزینه بالا رفته است ولی بقیه ویژگی ها تقریباً در حالت نرمال هستند. و در این سناریو هم دومین حالت بهترین حالت است. و نرخ جهش پایین کمک کرده که مسیر صاف بشود. و روش سوم بدترین حالت است.

سناریوی چهارم:



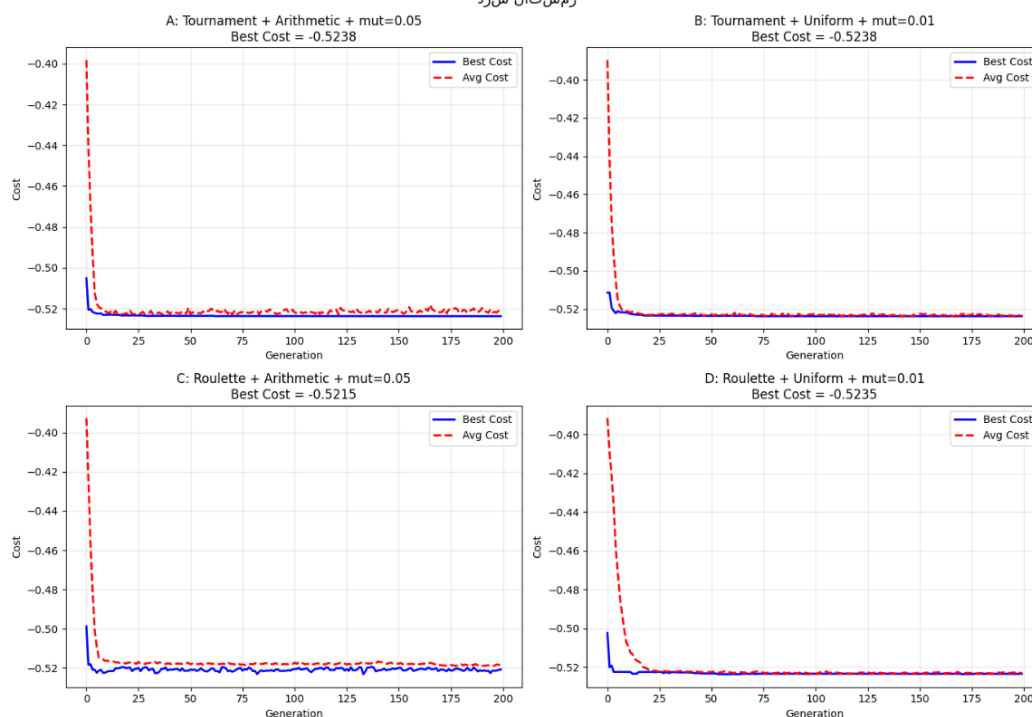
در این سناریو انتظار میرود که مصرف انرژی کاهش پیدا کند. و در این سناریو همه روش ها تقریباً شبیه به هم هستند زیرا قله های محلی کم هستند که به دلیل ساده تر بودن شرایط محیطی هستند.

سناریوی پنجم:



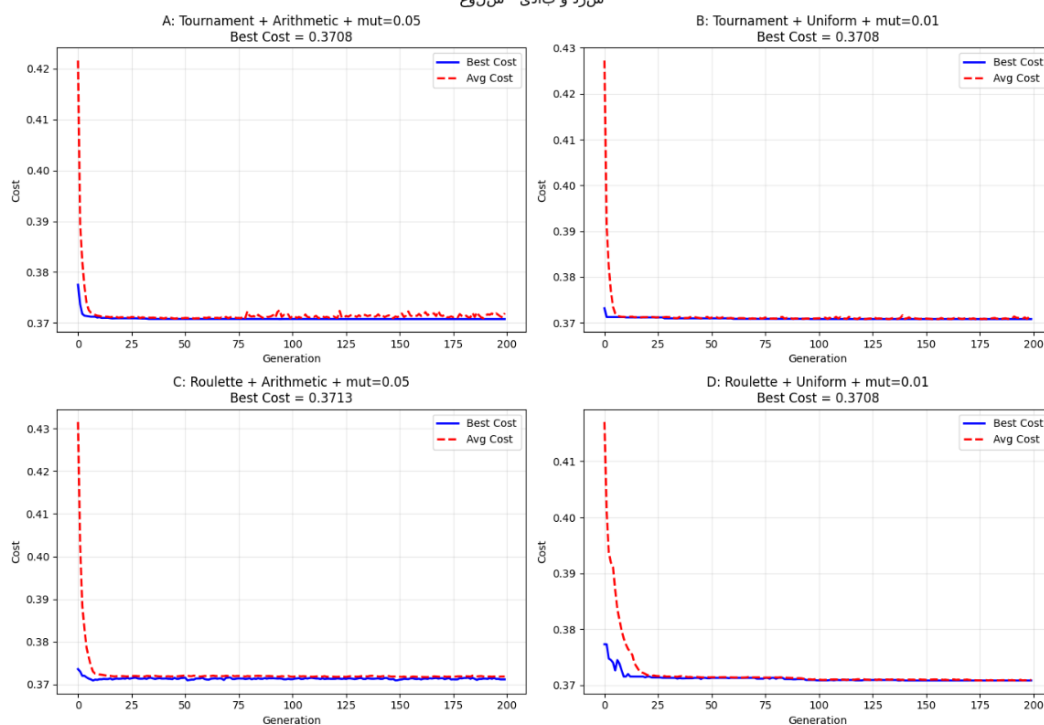
اینجا الگوریتم سریع بهینه شده ولی جواب مناسب نیست به دلیل اینکه ما قله های زیادی نداریم ولی حالت بهینه حالت خوبی نیست. که این به این دلیل است که سرعت باد بسیار بالا، رطوبت زیاد و تراکم متوسط افراد موجب افزایش مصرف انرژی تهویه و پنالتی های کنترل رطوبت شده اند. از طرف دیگر، به دلیل نبود بار حرارتی شدید، الگوریتم ژنتیک امکان کاهش چشمگیر مصرف انرژی و افزایش چشمگیر fitness را نداشته است.

سناریوی ۶:



با توجه به همگرایی سریع هر ۴ روش به این نتیجه میرسیم که قله های محلی جدی نداریم. و همینطور شرایط محیطی نسبتاً پایدار بوده و ساختمان با کمترین نوسان حرارتی مواجه است. و این نمودار نسبت به نمودار های قبلی نشان دهنده بالاترین سطح کارایی و آسایش است.

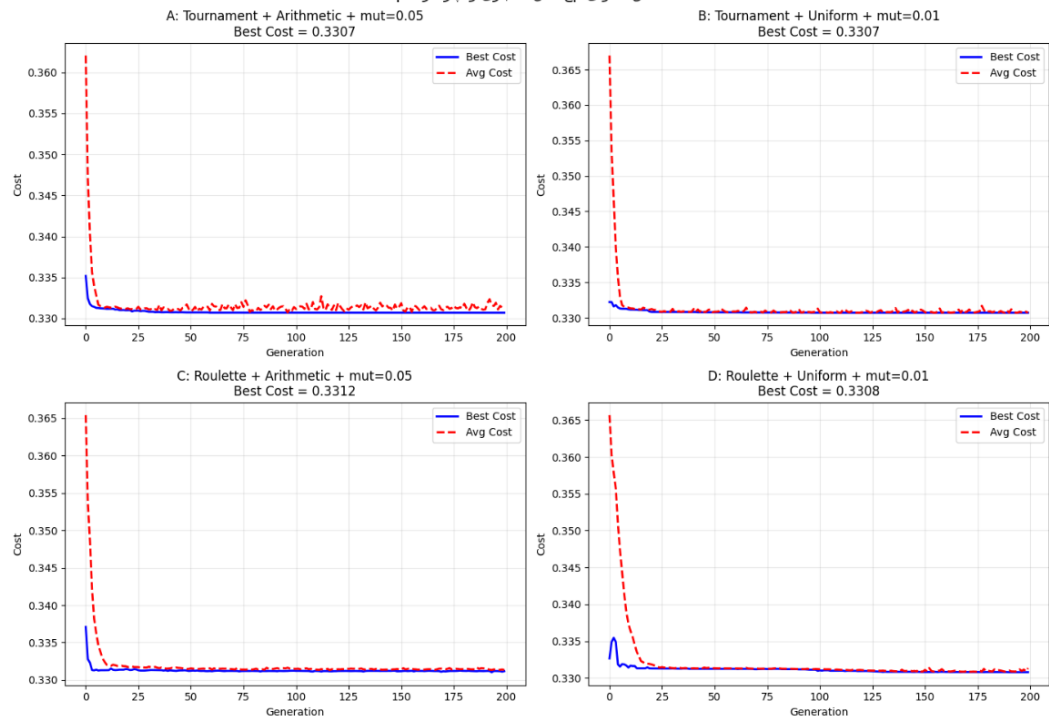
سناریوی هفتم:



هر ۴ نمودار افت سریع دارند. ولی در این سناریو تابع هزینه به نسبت سخت تر است و به این دلیل است که با سردتر شدن هوا و افزایش جمعیت تعادل بین آسایش و انرژی سخت تر میشود پس فیتنس کمتر و هزینه مثبت تر میشود.

سناریوی هشتم:

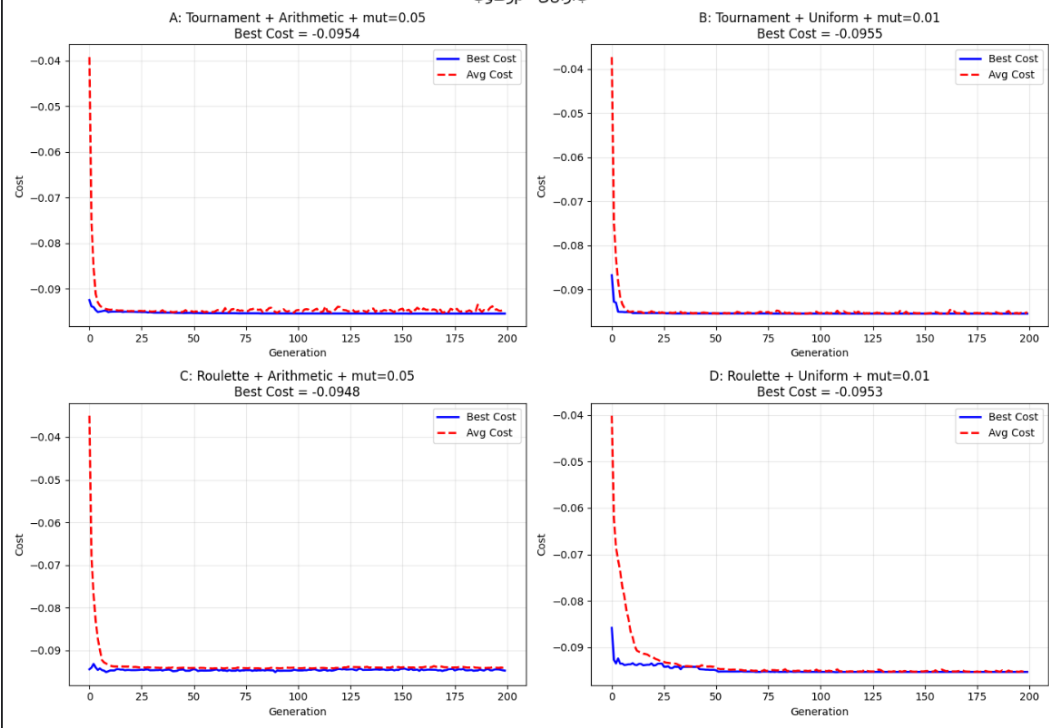
اتاق - هوای معتدل - ابری و پرتراکم



تابع هزینه در این سناریو نسبتاً هموار است ولی بهترین نقطه‌اش هم بد است چون شرایط محیطی خودشان افتضاح‌اند. هر ۴ روش تقریباً عدد مشابه دارند زیرا قله محلی جدی نداریم.

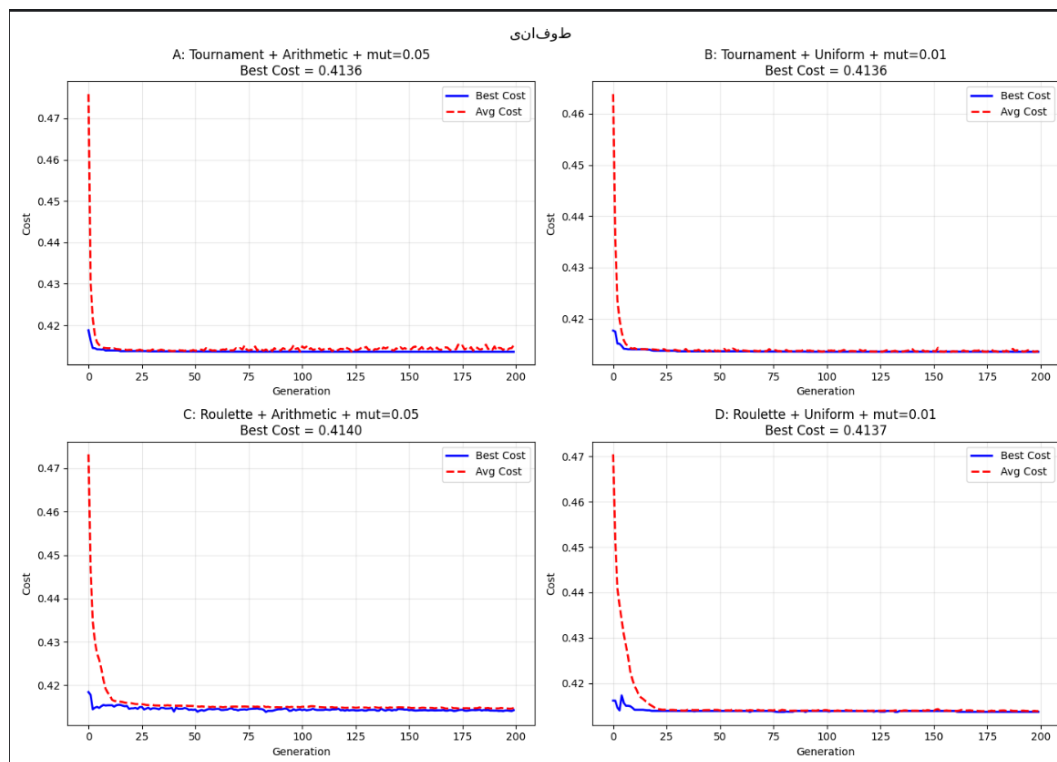
سناریوی نهم:

بارانی - مرطوب



در این سناریو تراکم کم است و باد متوسط و تابش خیلی زیاد و رطوبت کم و خشکی بسیار زیاد و فضای نسبتاً گرم داریم در اینجا چون فیتنس بهتر است هزینه نهایی منفی شده است. در اینجا با وجود تابش بسیار بالا (Solar=700) که منجر به افزایش بار حرارتی و محدود شدن بهبود انرژی می‌شود، تراکم کم (N=30) باعث کاهش پهنالی‌های کیفیت هوا و CO₂ شده است.

سناریوی دهم:



در اینجا به دلیل بالا بودن تراکم باعث می‌شود که مقدار $f1$ زیاد شده پس فیتنس کم بشود و هزینه بالا بره اما همگرایی خوب بوده و با توجه به کم بودن میزان نویز بهینه محلی زیاد نداشتیم.

تحلیل نمودار های $f1$, $f2$ تولید شده:

در فاز سوم تابع $f2$ در بعضی از موارد از همان نسل های ابتدایی تقریباً ثابت و اشباع می شود در حالی که $f1$ همچنان به تغییر و بهبود ادامه می دهد. که این اتفاق چند دلیل اصلی دارد. مهم ترین دلیلش به خاطر پله ای بودن توابع $f2$ است یعنی به عنوان مثال برای ST که امتیاز دما است ضریب دما از بازه ۲۰ تا ۲۴ ثابت است پس این باعث می شود که با تغییرات کوچک در دما، رطوبت و بقیه عامل های موثر در تابع $f2$ ضریب آن عامل تغییر چندانی نکند. یعنی به عنوان مثال دمای ۲۰ درجه با دمای ۲۱ درجه خروجی ثابتی می دهد و این باعث سریع اشباع شدن $f2$ می شود. اما در مقابل تابع $f1$ به صورت پیوسته تعریف شده و هر تغییر کوچکی در آن باعث تغییر در نمودار می شود.

همچنین در مقایسه با بخش دوم تابع $f2$ سریع تر اشباع می شود که به دلیل تعداد متغیر های بیشتر کروموزوم در بخش قبل می باشد. چون در بخش قبلی الگوریتم همزمان تمام متغیر های کنترلی را بهینه می کرد رسیدن به نقطه بهینه پایدار و ایده آل بیشتر طول میکشید. با ثابت شدن شرایط بیرونی، کار الگوریتم ژنتیک بسیار آسان تر می شود. ابعاد فضای جستجو به شدت کاهش یافته و الگوریتم خیلی سریع (حتی در همان نسل اول یا دوم) می تواند مقادیر بهینه ای برای μ پیدا کند که بهترین شرایط رشد گیاه را فراهم سازد.

پاسخ سوالات:

۱. کدام روش باعث همگرایی سریع تر شد؟ روش Tournament Selection باعث همگرایی سریع تر شد. به دلیل اینکه در نمودار های اول و دوم بهترین هزینه تقریباً بعد از ۵ تا ۱۰ نسل پیدا می شد. به دلیل اینکه افراد ضعیف سریع تر حذف می شوند و افراد خوب غالب می شوند.
۲. کدام روش جهش از گیر افتادن در مینیمم محلی جلوگیری کرد؟ روش جهش با نرخ ۰.۰۵ بیش از همه از گیر افتادن در مینیمم محلی جلوگیری کرد، چون تنوع ژنتیکی بیشتری ایجاد می کرد و فضای جست و جو را گسترده تر نگه می داشت.
۳. آیا Crossover از نوع Uniform باعث ایجاد تنوع بیشتری شد؟ به طور متوسط بله به دلیل اینکه این نوع تقطیع ژن ها را تصادفی تر ترکیب می کند. اما مقدار زیاد تنوع

اگر landscape حداقل واضح داشته باشد باعث اتلاف نسل های بی فایده میشود و منجر به بهبود نمیشود.

۴. آیا Tournament Selection باعث فشار انتخابی بیش از حد شد؟ فشار انتخابی یعنی شدت رقابت بین افراد برای بقا اگر این فشار خیلی زیاد باشد جمعیت خیلی سریع به چند فرد خاص محدود میشود و تنوع از بین میرود و ممکن است الگوریتم در حداقل محلی گیر کند. پس این روش ذاتاً فشار انتخابی بیشتری نسبت به روش رولت دارد. اما در این سناریو ها فشار انتخابی در حد مطلوب بود زیرا در غیر اینصورت شاهد نوسان یا گیر افتاد در حداقل محلی بودیم. اگر landscape پرنویز بود ممکن بود این اتفاق بیوفتد.

۵. آیا برخی ترکیب ها ناپایدار بودند؟ هیچ ترکیب ناپایداری مشاهده نشد.

بخش ۶:

پاسخ سوالات:

۱. آیا کاهش انرژی همیشه باعث افت آسایش شد؟ نه کاهش انرژی همیشه باعث افت آسایش نمی شود. ولی اگر کاهش انرژی از محدوده ی بهینه خارج شود، احتمال افت آسایش یا کیفیت رشد زیاد است چون بسیاری از پارامترهای رشد مثل دما، نور و CO2 خودشان مصرف انرژی دارند. اما اگر انرژی به شکل بهینه مدیریت شود (مثلاً تنظیم درست دما یا نور)، ممکن است مصرف انرژی کمتر شود و در عین حال کیفیت رشد تقریباً حفظ شود.

۲. حساسیت سیستم نسبت به وزن ها چقدر است؟

- تحلیل حساسیت ضرایب a_i و b_j (که در مرحله اول انجام شد) نشان داد که مدل به تغییرات کوچک (مثلاً ۵٪) در این ضرایب بسیار کم حساسیت است و تغییرات فیتنس نهایی

معمولاً کمتر از ۰.۰۲٪ بود. این پایداری بالا عمدتاً به دلیل نرمال‌سازی پویای f1_raw و f2_raw است.

- حساسیت به وزن‌های ترکیب w1 و w2: اگرچه در پروژه وزن‌ها ثابت در نظر گرفته شده‌اند و بهینه‌سازی آن‌ها منجر به نتایج خوبی نمی‌شود، اما سیستم نسبت به تغییرات این وزن‌ها حساسیت متوسطی دارد. افزایش وزن آسایش (مثلاً به ۰.۸) باعث می‌شود الگوریتم تمایل بیشتری به تنظیم دما، رطوبت، نور و CO₂ در محدوده ایده‌آل پیدا کند، حتی اگر مصرف انرژی افزایش یابد. برعکس، افزایش وزن انرژی باعث می‌شود مقادیر T_in به T_out نزدیک‌تر شده و نور مصنوعی کاهش یابد.

۳. کدام پارامتر GA بیشترین اثر را داشت؟

روش انتخاب (Selection method) بیشترین اثر را داشت. طبق نتایج مشاهده شده متوجه شدیم که انتخاب تورنومنت (با اندازه ۳) فشار انتخابی بالایی ایجاد کرد و به همگرایی سریع‌تر و مقادیر فیتنس منجر شد. ولی انتخاب چرخ رولت به دلیل فشار انتخابی کمتر، باعث شد الگوریتم دیرتر همگرا شود و گاهی در بهینه محلی گیر کند (به خصوص در ترکیب با تقاطع یکنواخت و نرخ جهش پایین).

همچنین نرخ جهش (Mutation rate) دومین اثر مهم بود. نرخ جهش ۰.۰۵ (بالا) در ترکیب با انتخاب تورنومنت و تقاطع حسابی، بهترین نتیجه را داد (همگرایی سریع و پایداری). اما نرخ جهش ۰.۰۱ (پایین) به خصوص در ترکیب با رولت و تقاطع یکنواخت، باعث کاهش تنوع و گیر افتادن در بهینه محلی شد (فیتنس نهایی پایین‌تر).

و در آخر روش تقاطع (Crossover method) تأثیر متوسط داشت. تقاطع حسابی (برای کدگذاری حقیقی) بسیار مناسب بود و به همگرایی هموار و جواب‌های با کیفیت منجر شد. تقاطع یکنواخت در اکثر موارد نتایج نزدیک به تقاطع حسابی داشت، اما در ترکیب با رولت و نرخ جهش پایین، عملکرد ضعیف‌تری نشان داد (نوسان و دیر همگرایی).

۴. آیا جواب نهایی یکتاست یا چند جواب تقریباً معادل وجود دارد؟ چند جواب تقریباً معادل وجود دارد. در این پروژه معمولاً به جای یک جواب مطلق، یک مجموعه جواب نزدیک

به بهینه داریم که بسته به وزن‌ها و تنظیمات GA ممکن است کمی فرق کنند مخصوصاً زمانی که نرخ جهش و روش سلکشن و تقطیع متفاوت می‌شد.